

What is 2D Array ?

two dimensional array is an array of arrays.

It stores data in rows and columns (like a table or matrix).

Example : 3 rows & 3 columns

```
1 2 3
4 5 6
7 8 9
```

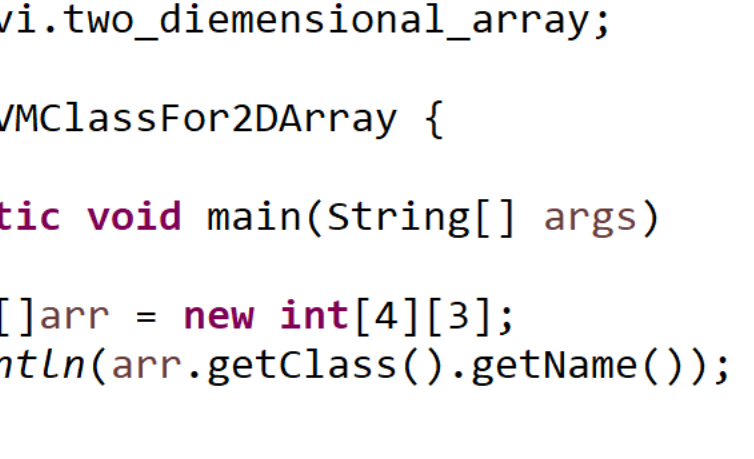
	Column	0	1	2
0		1 (0,0)	2 (0,1)	3 (0,2)
1		4 (1,0)	5 (1,1)	6 (1,2)
2		7 (2,0)	8 (2,1)	9 (2,2)
row				

* We need to use nested loop

* Outer loop will represent rows

* Inner loop will represent cols

How to define a 2D array :



```
package com.ravi.two_dimensional_array;
```

```
public class JVMClassFor2DArray {
```

```
{
```

```
    public static void main(String[] args)
```

```
    {
```

```
        int [][]arr = new int[4][3];
```

```
        IO.println(arr.getClass().getName()); //[[I
```

```
    }
```

```
}
```

```
public static String deepToString(Object [])
```

```
{
```

```
    //Predefined static method of Arrays class.
```

```
    * It is used to convert 2D array elements into String representation
```

```
package com.ravi.two_dimensional_array;
```

```
import java.util.Arrays;
```

```
//Using IO class
```

```
//WAP to print 2D Array element by using different ways [Dynamic input]
```

```
public class PrintingTwoDimensionalArray
```

```
{
```

```
    public static void main(String[] args)
```

```
    {
```

```
        int rows = Integer.parseInt(IO.readLine("Enter the number of rows :"));
```

```
        int cols = Integer.parseInt(IO.readLine("Enter the number of cols :"));
```

```
        int [][]arr = new int[rows][cols];
```

```
        for(int i=0; i<rows; i++) //rows
```

```
        {
```

```
            for(int j=0; j<cols; j++) //cols
```

```
            {
```

```
                arr[i][j] = Integer.parseInt(IO.readLine());
```

```
            }
```

```
        }
```

```
        IO.println("Printing the elements using ordinary for loop :");
```

```
        for(int i=0; i<rows; i++)
```

```
        {
```

```
            for(int j=0; j<cols; j++)
```

```
            {
```

```
                IO.print(arr[i][j]+" ");
```

```
            }
```

```
        } IO.println();
```

```
    }
```

```
    IO.println("Printing the elements using Arrays.deepToString() method");
```

```
    IO.println(Arrays.deepToString(arr));
```

```
    IO.println("Printing the elements using for each loop");
```

```
    for(int []x : arr)
```

```
    {
```

```
        IO.println(Arrays.toString(x));
```

```
    }
```

```
}
```

```
}
```

```
package com.ravi.two_dimensional_array;
```

```
import java.util.Arrays;
```

```
import java.util.InputMismatchException;
```

```
import java.util.Scanner;
```

```
public class PrintingTwoDimensionalArrayUsingScanner {
```

```
{
```

```
    public static void main(String[] args)
```

```
    {
```

```
        Scanner sc = new Scanner(System.in);
```

```
        try(sc)
```

```
        {
```

```
            IO.print("Enter the number of rows :");
```

```
            int rows = Integer.parseInt(sc.nextLine());
```

```
            IO.print("Enter the number of columns :");
```

```
            int cols = Integer.parseInt(sc.nextLine());
```

```
            int [][]numbers = new int[rows][cols];
```

```
            //Using for loop initialize the 2D array
```

```
            for(int i=0; i<rows; i++)
```

```
            {
```

```
                for(int j=0; j<cols; j++)
```

```
                {
```

```
                    numbers[i][j] = sc.nextInt();
```

```
                }
```

```
            }
```

```
            IO.println("Printing the elements :");
```

```
            IO.println(Arrays.deepToString(numbers));
```

```
        }
```

```
        catch(InputMismatchException e)
```

```
        {
```

```
            e.printStackTrace();
```

```
        }
```

```
    }
```

```
}
```

```
package com.ravi.two_dimensional_array;
```

```
import java.util.InputMismatchException;
```

```
import java.util.Scanner;
```

```
//WAP to search an element in the 2D array and print the index position of the element
```

```
public class ElementSearchIn2D {
```

```
{
```

```
    public static void main(String[] args)
```

```
    {
```

```
        Scanner sc = new Scanner(System.in);
```

```
        try(sc)
```

```
        {
```

```
            IO.print("Enter the number of rows :");
```

```
            int rows = sc.nextInt();
```

```
            IO.print("Enter the number of cols :");
```

```
            int cols = sc.nextInt();
```

```
            int [][]numbers = new int[rows][cols];
```

```
            //Initializing the 2D array
```

```
            for(int i=0; i<rows; i++)
```

```
            {
```

```
                for(int j=0; j<cols; j++)
```

```
                {
```

```
                    numbers[i][j] = sc.nextInt();
```

```
                }
```

```
            }
```

```
            IO.print("Enter the element to search :");
```

```
            int element = sc.nextInt();
```

```
            for(int i=0; i<rows; i++)
```

```
            {
```

```
                for(int j=0; j<cols; j++)
```

```
                {
```

```
                    if(element==numbers[i][j])
```

```
                    {
```

```
                        IO.println(element+" found at "+i+" rows and "+j+" cols");
```

```
                        break;
```

```
                    }
```

```
                }
```

```
            }
```

```
            catch(InputMismatchException e)
```

```
            {
```

```
                e.printStackTrace();
```

```
            }
```

```
        }
```

```
    }
```

```
}
```

```
package com.ravi.two_dimensional_array;
```

```
//Matrix to perform addition
```

```
public class MatrixForAddition {
```

```
{
```

```
    public static void main(String[] args)
```

```
    {
```

```
        int [][]x = {{1,2,7}, {3,4,1}}; //2 rows & 3 cols
```

```
        int [][]y = {{5,7,1}, {6,3,7}}; //2 rows & 3 cols
```

```
        int [][]sum = new int[2][3];
```

```
        for(int i=0; i<2; i++)
```

```
        {
```

```
            for(int j=0; j<3; j++)
```

```
            {
```

```
                sum[i][j] = x[i][j] + y[i][j];
```

```
            }
```

```
        }
```

```
        IO.println("Printing the Sum");
```

```
        for(int i=0; i<2; i++)
```

```
        {
```

```
            for(int j=0; j<3; j++)
```

```
            {
```

```
                IO.print(sum[i][j]+ " ");
```

```
            }
```

```
        } IO.println();
```

```
    }
```

```
}
```

```
}
```

```
Jagged Array :
```

A Jagged Array is a 2D array where each row can have a different number of columns.

	columns	0	1	2	3
0					
1					
2					

1st row -> 2 cols

2nd row -> 4 cols

3rd row -> 3 cols

```
package com.ravi.two_dimensional_array;
```

```
public class JaggedArray
```

```
{
```

```
    public static void main(String[] args)
```

```
    {
```

```
        int [][]arr = new int [3][]; //only number of rows are fixed
```

```
        arr[0] = new int[2]; //2 is number of column in 1st row
```

```
        arr[1] = new int[4]; //4 is number of column in 2nd row
```

```
        arr[2] = new int[3]; //3 is number of column in 3rd row
```

```
        arr[0][0] = 10;
```

```
        arr[0][1] = 20;
```

```
        arr[1][0] = 30;
```

```
        arr[1][1] = 40;
```

```
        arr[1][2] = 50;
```

```
        arr[1][3] = 60;
```

```
        arr[2][0] = 70;
```

```
        arr[2][1] = 80;
```

```
        arr[2][2] = 90;
```

```
        IO.println("Printing the Jagged array elements :");
```

```
        for(int i=0; i<3; i++)
```

```
        {
```

```
            for(int j=0; j<arr[i].length; j++)
```

```
            {
```

```
                IO.print(arr[i][j]+" ");
```

```
            }
```

```
        } IO.println();
```

```
    }
```

```
}
```

```
}
```

```
What is 3D Array :
```

A 3D Array (Three-Dimensional Array) is an array of 2D arrays.

Example : int[][][] arr; [Just like a cube]

```
package com.ravi.two_dimensional_array;
```

```
public class ThreeDimensionalArray
```

```
{
```

```
    public static void main(String[] args) {
```

```
        int[][][] arr = new int[2][2][2];
```

```
        int value = 1;
```

```
        // Initializing using loops
```

```
        for (int i = 0; i < arr.length; i++) // layers
```

```
        {
```

```
            for (int j = 0; j < arr[i].length; j++) // rows
```

```
            {
```

```
                for (int k = 0; k < arr[i][j].length; k++) // columns
```

```
                {
```

```
                    arr[i][j][k] = value++;
```

```
                }
```

```
            }
```

```
        }
```

```
        // Printing
```

```
        for (int i = 0; i < arr.length; i++) {
```

```
            System.out.println("Layer " + i);
```

```
            for (int j = 0; j < arr[i].length; j++) {
```

```
                for (int k = 0; k < arr[i][j].length; k++) {
```

```
                    System.out.print(arr[i][j][k] + " ");
```

```
                }
```

```
            } System.out.println();
```

```
        }
```

```
    }
```

```
}
```

```
Multithreading :
```

What is preemptive and non preemptive scheduling ?

Non Preemptive



* In non preemptive CPU scheduling, CPU cannot taken away without completing the assigned task

[p1 task is assigned so, first CPU will complete p1 then only we can assign another task]

Preemptive Scheduling :



* In preemptive scheduling, A CPU can taken away from one process to another process without completing the assigned task.

Uniprocessing :

* In Uniprocessing, We can assign only one task OR process to the CPU.

* It is a non preemptive scheduling.

Major drawbacks are :

a) Memory is wastage

b) Resources are wastage.

c) Most of the time, CPU is idle



* In order to avoid the drawback of Uniprocessing, We introduced multitasking OR Multiprocessing.

* It is a preemptive scheduling.

* Here our task will be completed as soon as possible

* Here CPU can switch (CPU Switching) from one process to another process frequently.



Multiprocessing

* Multitasking is divided into two types :

1) Process based multitasking

2) Thread based multitasking

Process Based Multitasking :



If a CPU will switch from one sub task (thread) of one process to another sub task of another process (costly operation, processes are heavy weight) then It is called Process Based Multitasking as shown in the above diagram.

Thread based multitasking



If a CPU will switch from one sub task of one process to another sub task of same process then it is called Thread based (light weight) multitasking as shown in the above diagram.